- Bcrypt Official Docs: https://github.com/dcodeIO/bcrypt.js
- More on CSRF Attacks: https://www.acunetix.com/websitesecurity/csrf-attacks/

---

in practice you should now choose a different package (https://www.npmjs.com/search?q=express%20csrf). You may, for example, consider using this package (which has a different API than `csurf()`): https://www.npmjs.com/package/csrf-csrf.

---

1. الـ Hashing vs Encryption

خد بالك إن الـ `bcrypt` بيعمل **Hashing** مش Encryption.

- **الـ Encryption:** طريق رايح جاي (ممكن تفك التشفير وترجع الباسورد الأصلي).
- **الـ Hashing:** طريق رايح بس (One-way function). مفيش أي طريقة رياضية ترجع الـ Hash ده لأصله. لما اليوزر يجي يعمل Login، إحنا بنعمل Hash للباسورد اللي دخله ونقارن الـ Hash بالـ Hash.

---

2. الـ "Salt Rounds" (رقم 12)

إنت كتبت `12` في الـ Parameter التاني، وده اختيار "لوريستي" جداً.

- **إيه ده؟** ده اسمه الـ **Cost Factor**. الـ `bcrypt` بيعمل عملية الـ Hashing دي 2^{12} مرة.
- **ليه 12؟** ده توازن مثالي بين الأمان والسرعة. لو الرقم قل (مثلاً 4)، الـ Hash هيكون سريع جداً والـ Hackers يقدروا يجربوا ملايين الباسورات في الثانية (Brute Force). لو زاد (مثلاً 20)، السيرفر هياخد وقت طويل جداً عشان يسجل يوزر واحد.

---

3. الـ "Salt" المستخبي

الـ `bcrypt` عبقري لأنه بيعمل حاجة اسمها **Salting** أوتوماتيك. هو بيضيف شوية حروف عشوائية للباسورد قبل ما يعمل له Hash.

- **الفائدة:** لو اتنين يوزرز اختاروا نفس الباسورد (مثلاً `123456`)، الـ Hash بتاع كل واحد هيكون مختلف تماماً في الداتابيز. ده بيمنع هجمات الـ **Rainbow Tables**.

---

4. تحليل كود:

```
// 1. bcrypt.hash is asynchronous and returns a promise
return bcrypt.hash(password, 12).then((cryptedPassword) => {
  // 2. cryptedPassword is a 60-character string starting with $2b$ or $2a$
  const user = new User({
    email: email,
    password: cryptedPassword, // No plain text anymore!
    cart: { items: [] },
  });
  return user.save();
})
```

---

1. الفكرة: الـ Custom Middleware

بدل ما نكتب `if (!req.session.isLoggedIn)` في كل Controller ونكرر الكود، بنعمل ملف Middleware صغير ونحقنه في الـ Routes اللي عايزين نحميها.

---

2. التنفيذ العملي (The Guard)

أول حاجة، هتعمل فولدر اسمه `middleware` وجواه ملف اسمه `is-auth.js`:

JavaScript

```
// middleware/is-auth.js
module.exports = (req, res, next) => {
    // لو اليوزر مش مسجل دخول، ارميه لصفحة الـ Login
    if (!req.session.isLoggedIn) {
        return res.redirect('/login');
    }
    // لو مسجل، كمل للـ Controller اللي بعده عادي
    next();
};
```

---

3. إزاي بنستخدمه في الـ Routes؟

دلوقتي هنروح لملف الـ Routes (مثلاً `routes/admin.js`) وننادي على البودي جارد ده:

JavaScript

```
const express = require('express');
const isAuth = require('../middleware/is-auth'); // استدعاء البودي جارد
const adminController = require('../controllers/admin');

const router = express.Router();

// بنحط الـ isAuth قبل الـ Controller
// التنفيذ بيمشي من الشمال لليمين
router.get('/add-product', isAuth, adminController.getAddProduct);
router.post('/add-product', isAuth, adminController.postAddProduct);

module.exports = router;
```

---

1. `app.use(csrfProtection)`

ده الميدل وير الأساسي. وظيفته إنه بيراقب كل الـ **Non-GET Requests** (زي POST, PATCH, DELETE).

- **إزاي بيفكر؟** أول ما يوصل طلب `POST` مثلاً، بيروح يدور في الـ Body أو الـ Headers على حاجة اسمها `_csrf` .
- **القرار:** لو لقى التوكين ومطابق للي متسيف في السيشين، بيقوله "اتفضل يا باشا" وينادي على `next()` . لو ملقاهوش أو لقى توكين غلط، بيضرب Error فوراً ويوقف الطلب.

2. `res.locals` (الـ Views البوسطجي بتاع الـ)

الـ `res.locals` ده Object عبقري في Express. أي حاجة بتحطها جواه بتبقى متاحة **أوتوماتيك** في كل ملفات الـ **EJS** (أو الـ View engine) اللي هتعمل لها `render` في الـ Request ده.

- بدل ما تبعت الـ `csrfToken` يدوي في كل Controller (زي `res.render('shop', {token: ...})`)، أنت هنا بعته "مرة واحدة وللكل".

3. `res.locals.csrfToken = req.csrfToken()`

الميثود `req.csrfToken()` دي جاية من مكتبة `csurf` .

- **وظيفتها:** بتكريت "سلسلة حروف عشوائية" (Unique String) وتخزنها في السيشين، وفي نفس الوقت بتجهزها عشان تتبعت للـ View.
- **السر:** التوكين ده بيتغير مع كل Request أو مع كل Session جديدة، فمستحيل الهكر يتوقعه.

---

1. إيه هي الـ Flash Message أصلاً؟

هي عبارة عن داتا (غالباً رسالة نصية) بتتخزن في السيشين **لمدة طلب واحد بس** (One Request Life-cycle). أول ما السيرفر يقرأها عشان يعرضها لليوزر، بيمسحها أوتوماتيك من السيشين.

---

2. الميكانيزم الداخلي (How it works internally)

إحنا بنستخدم مكتبة اسمها `connect-flash`. تعال نشوف بتعمل إيه ورا الكواليس:

1. **التخزين (Writing):** لما بتنادي `req.flash('error', 'Invalid email')`.. المكتبة دي بتروح للـ `req.session` وتزود Object جواه اسمه `flash`.
2. **الـ Persistence:** السيشين بتتسيف في المونجو (عن طريق الـ Store اللي عملناه)، والرسالة بتفضل مستنية.
3. **القراءة والمسح (Reading & Deleting):** لما بتنادي `req.flash('error')` في الـ Controller اللي بيعرض الصفحة:
   - المكتبة بتجيب الرسالة من السيشين وتديها لك.
   - **فوراً** بتمسح الرسالة دي من الـ `req.session` في الداتابيز.
   - عشان كدة لو اليوزر عمل Refresh، الـ `req.flash('error')` هترجع `[]` (Array فاضي).

---

3. التنفيذ العملي (The Setup)

أ- التعريف في `app.js` :

JavaScript

```
const flash = require('connect-flash');

// لازم بعد الـ session وقبل الـ routes
app.use(flash());
```

ب- الاستخدام في الـ Controller (الـ Signup مثلاً):

JavaScript

```
exports.postSignup = (req, res, next) => {
  User.findOne({ email: email }).then(userDoc => {
    if (userDoc) {
      // بنخزن الرسالة في السيشين قبل ما نعمل ريدايركت
      req.flash('error', 'الايميل ده محجوز يا برنس، جرب واحد تاني.');
      return res.redirect('/signup');
    }
    // ... كمل كود التشفير والحفظ
```

```
  });
};
```

ج- العرض في الـ View (الـ EJS):

لازم الأول نسحب الرسالة من السيشين ونبعتها للـ Template:

JavaScript

```
exports.getSignup = (req, res, next) => {
  // هنا المونجووس بيمسح الرسالة من الداتابيز بمجرد القراءة
  let message = req.flash('error');
  if (message.length > 0) {
    message = message[0];
  } else {
    message = null;
  }
  res.render('auth/signup', {
    errorMessage: message
  });
};
```

---

4. ليه بنستخدمها وما بنبعتش الرسالة في الـ Redirect؟

الـ `res.redirect` مش بياخد داتا معاه (لأنه بيخلي المتصفح يبعت `GET` request جديد خالص). الطريقة الوحيدة عشان "تجرجر" داتا معاك من `POST` لـ `GET` هي إنك تحطها في "الأمانات" (السيشين) والـ Flash هو اللي بينظم عملية الاستلام والتسليم دي.